

Problem Set 0: RAW Image Processing Pipeline

Ata Dural

June 7, 2026

1 Introduction

The goal of this project was to take a RAW camera image and turn it into a normal color image that can be viewed on a screen. The input file was `banana_slug.tiff`. At the start, this image is just sensor data, so it does not look like a regular photo yet.

For this project, I wrote a MATLAB script that follows the main steps of a simple image processing pipeline. My code loads the image, fixes the pixel range, separates the Bayer color pattern, applies white balancing, fills in the missing color values, adjusts brightness, applies gamma correction, and saves the final image. I also compared PNG and JPEG file sizes to see how much compression changed the output.

2 Methods

2.1 Loading the Image

I first loaded the RAW image in MATLAB using `imread`. The image was stored as a two dimensional array with unsigned 16 bit integer values. I used `size` and `class` to check the image information.

The image had the following properties:

- Height: 2856 pixels
- Width: 4290 pixels
- Data type: `uint16`

After checking the image, I converted it to `double`. This made it easier to scale the pixel values and do the rest of the calculations.

2.2 Linearization

The original pixel values were not ready to display directly. The image had a black level of 2047 and a saturation level of 15000. I used these two values to map the image into the range from 0 to 1.

$$I_{\text{linear}} = \frac{I_{\text{raw}} - 2047}{15000 - 2047}$$

After this step, I clipped the values so that anything below 0 became 0 and anything above 1 became 1. This helped remove values below the black level and limit overexposed values.

2.3 Bayer Pattern

The RAW image does not store a full red, green, and blue value at every pixel. Instead, each pixel only stores one color value. I used the **rggb** Bayer pattern:

$$\begin{bmatrix} R & G \\ G & B \end{bmatrix}$$

Using this pattern, I separated the image into four groups:

- Red pixels from odd rows and odd columns
- First green pixels from odd rows and even columns
- Second green pixels from even rows and odd columns
- Blue pixels from even rows and even columns

2.4 White Balancing

I implemented both Gray World and White World white balancing.

For Gray World, I scaled the red and blue values so that their average values matched the average green value. I used green as the reference because the Bayer pattern has two green pixels in every block.

$$R' = R \cdot \frac{G_{\text{avg}}}{R_{\text{avg}}}$$

$$B' = B \cdot \frac{G_{\text{avg}}}{B_{\text{avg}}}$$

For White World, I scaled the red and blue values so that their maximum values matched the maximum green value.

$$R' = R \cdot \frac{G_{\text{max}}}{R_{\text{max}}}$$

$$B' = B \cdot \frac{G_{\text{max}}}{B_{\text{max}}}$$

I used Gray World for the final image. It looked more neutral to me. White World worked too, but it made the image look warmer and more yellow.

2.5 Demosaicing

After white balancing, I needed to fill in the missing color values. This step is called demosaicing. I used MATLAB's **interp2** function to do bilinear interpolation.

The red and blue channels were each on their own regular grid, so I interpolated them directly. The green channel had two separate grids, so I interpolated both of them and then averaged the results. After that, I combined the red, green, and blue channels into one RGB image.

2.6 Brightness and Gamma Correction

The image was still too dark after demosaicing, so I adjusted the brightness. I converted the image to grayscale and scaled the image so that the mean grayscale value became 25 percent.

The brightness scale factor from my MATLAB output was:

$$4.30$$

Then I applied the sRGB gamma correction curve:

$$C' = \begin{cases} 12.92C, & C \leq 0.0031308 \\ 1.055C^{1/2.4} - 0.055, & C > 0.0031308 \end{cases}$$

Here, C is a color channel value from the image.

2.7 Compression

At the end, I saved the final processed image as a PNG file and also as JPEG files with different quality values. I used JPEG quality values of 95, 75, 50, 25, and 10. Then I compared the file sizes and calculated the compression ratio between the PNG image and the JPEG image at quality 95.

3 Results

3.1 Final Processed Image

Figure 1 shows the final image after the full pipeline. This version uses Gray World white balancing.



Figure 1: Final processed image using Gray World white balancing.

3.2 White Balance Comparison

Figure 2 shows the Gray World and White World results. I applied the same brightness and gamma correction to both images so the comparison would be fair.



Figure 2: Gray World and White World white balance comparison.

Gray World looked better for the final image because the colors looked more natural. White World made the image look warmer, especially in the dirt and background.

3.3 MATLAB Output

The main output values from MATLAB are shown in Table 1.

Item	Value
Image height	2856 pixels
Image width	4290 pixels
Image type	uint16
Brightness scale factor	4.30
Target mean grayscale value	25 percent

Table 1: Main image information and brightness value.

3.4 Compression Results

The file sizes from the compression test are shown in Table 2.

File	Size
PNG	17.21 MB
JPEG quality 95	3.38 MB
JPEG quality 75	1.29 MB
JPEG quality 50	0.85 MB
JPEG quality 25	0.56 MB
JPEG quality 10	0.35 MB

Table 2: PNG and JPEG file sizes.

The compression ratio between the PNG file and the JPEG quality 95 file was:

$$\text{Compression Ratio} = \frac{17.21}{3.38} = 5.10$$

This means the JPEG quality 95 image was about 5.10 times smaller than the PNG image.

From my visual check, JPEG quality 75 was the lowest setting that still looked very close to the PNG version. The lower quality images were smaller, but the image quality started to drop more.

4 Brief Conclusion

In this project, I built a RAW image processing pipeline in MATLAB. The code loaded the RAW image, linearized the pixel values, used the `rggb` Bayer pattern, applied white balancing, demosaiced the image, adjusted brightness, applied gamma correction, and saved the results.

The final image looked like a normal color photo after all of these steps. I chose Gray World white balancing because it gave a more neutral result than White World. The compression test also showed that JPEG can reduce the file size a lot. At quality 95, the JPEG file was about 5.10 times smaller than the PNG file. One limitation is that some choices, like brightness and white balance, depend on what looks best visually.